

RELATÓRIO – BANCO DE DADOS DE VENDAS

Projeto de Banco de Dados Relacional

1. Introdução

Este relatório apresenta o desenvolvimento de um banco de dados criado para uma empresa que precisava organizar melhor suas informações de vendas. Antes da criação do sistema, os dados estavam espalhados em planilhas, o que podia dificultar consultas, gerar informações repetidas e até causar erros nos relatórios.

A ideia do projeto foi criar um banco de dados relacional capaz de guardar informações de clientes, produtos e pedidos de forma organizada. Com isso, a empresa consegue consultar os dados com mais facilidade e ter uma visão melhor das suas vendas.

Durante o projeto foram utilizadas tabelas relacionadas por chaves primárias e estrangeiras. Também foram estudados recursos importantes de bancos de dados, como views, procedures e triggers. Esses recursos ajudam a deixar o sistema mais organizado e podem automatizar algumas tarefas.

2. Objetivo do projeto

O principal objetivo foi desenvolver uma estrutura simples, mas que pudesse representar uma situação real de uma loja. O banco deveria permitir cadastrar clientes e produtos, registrar pedidos e fazer consultas para descobrir informações importantes sobre as vendas.

Entre as consultas propostas estão descobrir quais clientes compraram mais, qual produto foi mais vendido em determinado mês, quanto cada cliente gastou no total e quais pedidos ultrapassaram um valor definido.

3. Tipo de banco de dados utilizado

O banco de dados utilizado no projeto é do tipo relacional. Nesse modelo, as informações são organizadas principalmente em tabelas e cada tabela possui campos que representam as características dos dados. As tabelas podem ser relacionadas entre si por meio de chaves.

O modelo relacional foi escolhido porque combina bem com o problema da empresa. Como existem clientes, produtos e pedidos, é necessário saber quais informações pertencem a cada parte e como elas se relacionam. Por exemplo, um pedido precisa estar ligado a um cliente e também a um produto.

Outra vantagem é que o modelo relacional ajuda a evitar a repetição de informações. Em vez de colocar o nome completo do cliente em todos os pedidos, o pedido guarda o seu código,

chamado de chave estrangeira. Esse código aponta para o cadastro correto na tabela de clientes.

3.1 Por que usar um banco relacional?

- Organiza os dados em tabelas.
- Permite relacionar informações de diferentes tabelas.
- Ajuda a evitar dados repetidos.
- Facilita a realização de consultas.
- Permite controlar melhor a integridade dos dados.
- Pode ser utilizado em sistemas reais de lojas e empresas.

4. Estrutura geral do banco

A estrutura criada possui três tabelas principais: clientes, produtos e pedidos. A tabela clientes armazena as pessoas que compram na loja. A tabela produtos guarda os produtos disponíveis para venda. Já a tabela pedidos registra as compras realizadas.

A relação principal é entre clientes e pedidos e entre produtos e pedidos. Um cliente pode fazer vários pedidos, enquanto cada pedido pertence a um cliente. Da mesma forma, um produto pode aparecer em vários pedidos.

Essa organização deixa o banco mais fácil de entender e também facilita a criação de consultas usando comandos SQL como SELECT, JOIN, GROUP BY e ORDER BY.

5. Tabelas do banco de dados

5.1 Tabela clientes

A tabela clientes foi criada para armazenar os dados básicos das pessoas que realizam compras. Ela possui o campo `id_cliente`, que é a chave primária e identifica cada cliente de forma única.

- `id_cliente`: código único do cliente.
- `nome`: nome do cliente.
- `email`: endereço de e-mail.
- `telefone`: telefone para contato.

Com essa tabela é possível consultar os clientes cadastrados e relacioná-los aos pedidos feitos por cada um.

5.2 Tabela produtos

A tabela produtos armazena os produtos que a loja vende. Também possui uma chave primária, chamada `id_produto`. Além dela, existem informações como nome, descrição, preço e estoque.

- `id_produto`: código único do produto.
- `nome`: nome do produto.

- descrição: explicação ou características do produto.
- preço: valor de venda do produto.
- estoque: quantidade disponível.

Essa tabela é importante porque permite saber quais produtos existem na loja, quanto custam e qual quantidade está disponível.

5.3 Tabela pedidos

A tabela pedidos registra as compras. Ela possui o `id_pedido` como chave primária. Também possui `id_cliente` e `id_produto`, que são chaves estrangeiras e fazem a ligação com as outras tabelas.

- `id_pedido`: identifica cada pedido.
- `id_cliente`: identifica o cliente que fez a compra.
- `id_produto`: identifica o produto comprado.
- `quantidade`: quantidade comprada.
- `data_pedido`: data em que a compra foi realizada.

O campo `id_cliente` aponta para a tabela clientes e o campo `id_produto` aponta para produtos. Assim, o banco consegue saber quem comprou, qual produto foi comprado, a quantidade e a data.

5

6. Consultas SQL e resultados

Uma das partes mais importantes do projeto foi criar consultas que transformassem os dados armazenados em informações úteis para a empresa. Apenas guardar os dados não é suficiente: também é necessário conseguir pesquisar e analisar essas informações.

6.1 Clientes que realizaram mais compras

Essa consulta utiliza `COUNT` para contar a quantidade de pedidos de cada cliente. Depois, os resultados são organizados do maior para o menor. Dessa forma, a empresa consegue identificar os clientes que mais compram.

6.2 Produto mais vendido no mês

Para descobrir o produto mais vendido, é utilizada a quantidade registrada nos pedidos. A consulta também pode filtrar o mês e o ano. Depois de somar as quantidades, o produto com maior resultado aparece como o mais vendido.

6.3 Valor total de vendas por cliente

Nessa consulta é feita uma conta usando a quantidade comprada multiplicada pelo preço do produto. Depois, os valores são agrupados por cliente. Isso permite saber quanto cada pessoa gastou no total.

6.4 Pedidos acima de determinado valor

Essa consulta permite colocar um limite de valor. No exemplo do projeto, foram procurados pedidos acima de R\$ 500,00. Esse tipo de consulta pode ser útil para identificar compras de maior valor ou analisar clientes que fazem pedidos maiores.

7. Normalização

A normalização é uma forma de organizar o banco para diminuir repetições e problemas nos dados. Neste projeto, os dados foram separados de acordo com sua função. Os dados dos clientes ficam em clientes, os produtos ficam em produtos e as informações das compras ficam em pedidos.

Por exemplo, se um cliente fizer várias compras, seu nome não precisa ser repetido em todos os registros. O pedido utiliza apenas o id_cliente para fazer a ligação. Isso deixa o banco mais organizado e facilita alterações futuras.

8. Procedures: o que são e para que servem?

Uma stored procedure, ou procedimento armazenado, é um conjunto de comandos SQL que fica salvo dentro do banco de dados. Ela pode ser chamada sempre que determinada tarefa precisar ser executada. Em vez de escrever os mesmos comandos várias vezes, podemos criar uma procedure e reutilizá-la.

No projeto da loja, uma procedure poderia ser criada para cadastrar um novo pedido. Ela receberia o código do cliente, o código do produto e a quantidade. Depois poderia inserir o pedido e realizar outras verificações necessárias.

Outra possibilidade seria criar uma procedure para consultar as vendas de um cliente específico. Isso seria interessante porque a mesma consulta poderia ser utilizada várias vezes, mudando somente o cliente informado.

Exemplo de uso

Uma procedure para cadastrar pedidos poderia receber parâmetros como id_cliente, id_produto e quantidade. Ao ser executada, ela faria o INSERT na tabela pedidos. Em um sistema maior, ela também poderia verificar se o produto existe e se existe quantidade suficiente em estoque.

9. Views: o que são e quais dados podem mostrar?

Uma view é como uma consulta SQL salva dentro do banco de dados. Ela não precisa criar uma nova tabela com os mesmos dados. Em vez disso, guarda uma consulta que pode ser acessada depois como se fosse uma tabela.

Para este banco, uma view interessante seria uma visão de vendas, juntando clientes, pedidos e produtos. Ela poderia mostrar o nome do cliente, o produto comprado, a quantidade, o preço e o valor total da compra.

Também poderia existir uma view mostrando o total gasto por cliente. Isso facilitaria a criação de relatórios, porque a empresa não precisaria montar a mesma consulta toda vez.

Exemplo de dados de uma View de vendas

- Nome do cliente
- Nome do produto
- Quantidade comprada
- Preço do produto
- Data do pedido
- Valor total do pedido

As views são úteis principalmente quando determinadas informações são consultadas com frequência. Elas também podem deixar consultas mais complicadas mais fáceis de utilizar.

10. Triggers: o que fazem?

Trigger, ou gatilho, é um recurso do banco de dados que executa uma ação automaticamente quando acontece determinado evento em uma tabela. Os eventos mais comuns são INSERT, UPDATE e DELETE.

No banco da loja, um exemplo interessante seria usar uma trigger para atualizar o estoque automaticamente quando um pedido fosse registrado. Quando uma venda fosse inserida, a quantidade vendida poderia ser diminuída do estoque do produto.

Outra possibilidade seria criar uma trigger para impedir uma venda quando não houver estoque suficiente. Assim, o banco ajudaria a evitar um erro no cadastro de pedidos.

Exemplo de funcionamento

Imagine que existem 10 unidades de um produto no estoque e um cliente compra 3. Ao registrar o pedido, uma trigger poderia diminuir automaticamente o estoque para 7. Dessa forma, não seria necessário fazer essa alteração manualmente toda vez.

É importante tomar cuidado com triggers, porque elas executam ações automaticamente. Se forem criadas de forma errada, podem causar alterações inesperadas. Por isso, devem ser utilizadas quando realmente ajudam no funcionamento do sistema.

11. Conclusão

O desenvolvimento desse banco de dados mostrou como o SQL pode ser usado para organizar informações de uma empresa. Com as tabelas de clientes, produtos e pedidos, foi possível representar uma situação parecida com a de uma loja real.

Também foi possível entender a importância das chaves primárias e estrangeiras para criar relações entre as tabelas. As consultas permitem transformar os dados armazenados em informações úteis para a tomada de decisões.

Além das tabelas e consultas, recursos como procedures, views e triggers podem deixar o sistema mais completo. Procedures ajudam a reutilizar tarefas, views facilitam consultas e relatórios, e triggers permitem automatizar ações quando determinados eventos acontecem.

No final, o banco de dados atende à proposta inicial de centralizar as informações e facilitar o controle das vendas. Mesmo sendo um projeto de estudo, sua estrutura pode servir como base para um sistema maior, com mais tabelas, usuários, relatórios e funções.

12. Considerações finais

Este projeto também ajudou a entender que criar um banco de dados não é apenas escrever comandos SQL. É preciso pensar em como os dados serão organizados, como as tabelas vão se relacionar e quais informações serão necessárias no futuro. Uma boa organização desde o começo pode evitar vários problemas quando o sistema crescer.